

SKRIPSI

**ANALISIS EFEKTIVITAS ROLE-BASED MULTI-AGENT LLM DALAM
DETEKSI AMBIGUITAS REQUIREMENT PERANGKAT LUNAK**

***ANALYZING THE EFFECTIVENESS OF ROLE-BASED MULTI-AGENT
LLM FOR SOFTWARE REQUIREMENT AMBIGUITY DETECTION***

Usulan Penelitian



ANDREANDHIKI RIYANTA PUTRA
23/517511/PA/22191

**PROGRAM STUDI ILMU KOMPUTER
DEPERTEMEN ILMU KOMPUTER DAN ELEKTRONIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS GADJAH MADA
YOGYAKARTA**

2026

Daftar Isi

Halaman Judul	i
I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Perumusan Masalah	2
1.3 Batasan Penelitian	3
1.4 Tujuan Penelitian	3
1.5 Manfaat Penelitian	4
II PENELITIAN TERKAIT	5
2.1 Studi Terkait	5
2.1.1 Deteksi Ambiguitas Requirement Menggunakan Single-Agent LLM	5
2.1.2 Multi-Agent LLM untuk Requirements Engineering	5
2.1.3 Multi-Agent LLM di Domain Lain yang Relevan	6
2.1.4 Fondasi Multi-Agent LLM dan Tantangan Self-MoA	6
2.1.5 Sintesis dan Identifikasi Gap	7
III METODE DAN RANCANGAN PENELITIAN	8
3.1 Rancangan Penelitian	8
3.1.1 Tahap 1: Pengumpulan dan Preparasi Dataset	8
3.1.2 Tahap 2: Perancangan Arsitektur Role-Based Multi-Agent LLM	8
3.1.3 Tahap 3: Konfigurasi Komparatif	9
3.1.4 Tahap 4: Pengukuran dan Evaluasi	9
3.1.5 Tahap 5: Analisis Lintas Bahasa	10
3.2 Alat dan Bahan	10
IV JADWAL PENELITIAN	12
4.1 Rencana Kerja	12
GLOSARIUM	13
DAFTAR PUSTAKA	13

Bab I

PENDAHULUAN

1.1 Latar Belakang

Ambiguitas dalam *requirement* perangkat lunak merupakan penyebab utama kegagalan proyek pengembangan perangkat lunak, yang berkontribusi pada lebih dari 70% kasus kegagalan proyek The Standish Group (1995); Wiegers (2003). Ambiguitas terjadi ketika suatu *requirement* dapat ditafsirkan lebih dari satu cara oleh pihak yang berbeda—misalnya *stakeholder*, pengembang, dan penguji—sehingga mengakibatkan kesalahpahaman yang merambat ke seluruh siklus pengembangan, mulai dari desain arsitektur yang keliru hingga *rework* yang mahal pasca-implementasi Berry et al. (2000); Kamsties et al. (1998). Taksonomi ambiguitas yang telah mapan dalam literatur mencakup enam kategori utama: ambiguitas leksikal, sintaktis, semantik, kekaburan (*vagueness*), ketidaklengkapan (*incompleteness*), dan kontradiksi Berry and Kamsties (2004); Kamsties et al. (1998). Setiap kategori memerlukan lensa analisis yang berbeda untuk dapat terdeteksi secara efektif.

Upaya otomatisasi deteksi ambiguitas *requirement* telah berkembang dari pendekatan berbasis aturan (*rule-based*) seperti *Natural Language Inference* (NLI) tradisional Ferrara et al. (2014) menuju pemanfaatan *Large Language Models* (LLM) sebagai *single-agent* Bashir et al. (2025). Bashir et al. (2025) pada studi industri di ICSME menunjukkan bahwa LLM *single-agent* mampu meningkatkan deteksi ambiguitas sebesar +20,2% dengan konfigurasi *10-shot prompting*, dan mendapatkan skor persetujuan manusia (*human agreement*) sebesar 3,84 dari 5 Bashir et al. (2025). Meskipun hasil ini menjanjikan, pendekatan *single-agent* memiliki keterbatasan fundamental: LLM tunggal membaca *requirement* dari satu perspektif saja dan tidak secara alami mengadopsi sudut pandang berbeda yang diperlukan untuk menangkap seluruh jenis ambiguitas Bashir et al. (2025); Vijayvargiya et al. (2025). Sebagai contoh, seorang pengembang mungkin mendeteksi ambiguitas teknis yang terlewat oleh penguji, dan sebaliknya—setiap peran memiliki kepekaan yang berbeda terhadap jenis ambiguitas tertentu.

Paradigma *multi-agent* dalam kerangka kerja *Mixture-of-Agents* (MoA) Wang et al. (2024) telah menunjukkan potensi signifikan dalam meningkatkan kapabilitas LLM melalui orkestrasi beberapa agen. Dalam domain *Requirements Engineering* (RE), beberapa penelitian terkini telah mengadopsi pendekatan ini: Oriol et al. (2025) mengaplikasikan strategi *Multi-Agent Debate* (MAD) untuk klasifikasi fungsional vs. non-fungsional dengan peningkatan F1 sebesar +10,9% dibandingkan *baseline*, namun biaya komputasi meningkat 16 kali lipat dan pendekatan mereka bersifat *debater-vs-debater* tanpa spesialisasi peran Oriol et al. (2025); Jin et al. (2025) mengusulkan iReDev dengan 6 agen berbasis peran (*interviewer*, *end-user*, *deployer*, *analyst*, *archivist*, *reviewer*) untuk siklus RE end-to-end, tetapi tidak secara khusus mengevaluasi kontribusi tiap peran terhadap deteksi ambiguitas Jin et al. (2025); Cheng et

al. (2026) mengembangkan QUARE dengan agen terkualitas dan negosiasi dialektis untuk atribut kualitas dengan kepatuhan 98,2%, namun fokusnya bukan pada ambiguitas Cheng et al. (2026); dan Lu et al. (2026) membangun ReDeFo dengan agen Analyst, Formalizer, dan Coder untuk formalisasi *requirement* ke kode terverifikasi, bukan untuk identifikasi ambiguitas Lu et al. (2026).

Kesenjangan penelitian (*research gap*) yang teridentifikasi adalah: *belum ada penelitian yang secara spesifik menguji arsitektur role-based multi-agent LLM untuk deteksi ambiguitas requirement perangkat lunak*. Yang paling mendekati adalah Oriol et al. (2025) dengan *debate* antar-agen tanpa peran, dan Bashir et al. (2025) dengan LLM tunggal tanpa orkestrasi multi-agent. Kedua pendekatan ini tidak mengeksploitasi prinsip bahwa ambiguitas bersifat *multi-perspectival*—yaitu, ambiguitas yang tidak terdeteksi dari satu sudut pandang mungkin terlihat jelas dari sudut pandang lain. Dengan mendesain agen-agen yang masing-masing mengemban peran *stakeholder*, *developer*, dan *tester*, sistem diharapkan dapat menangkap spektrum ambiguitas yang lebih luas dibandingkan pendekatan *single-agent* maupun *multi-agent* tanpa spesialisasi peran.

Namun, pendekatan *multi-agent* juga menghadapi tantangan yang perlu direspons secara empiris. Li et al. (2025) menunjukkan bahwa konfigurasi *Self-MoA*—di mana model yang sama diulang beberapa kali—seringkali mengungguli *Mixed-MoA* yang menggunakan model berbeda Li et al. (2025). Temuan ini menimbulkan pertanyaan kritis: *apakah spesialisasi peran (role-based) memberikan nilai tambah yang signifikan dibandingkan sekadar mengulang model yang sama?* Penelitian ini dirancang untuk menjawab pertanyaan tersebut melalui studi komparatif yang mencakup tiga konfigurasi: (1) *single-agent LLM*, (2) *Self-MoA*, dan (3) *role-based multi-agent*.

Selain itu, dimensi bahasa menjadi diferensiator yang penting. Sebagian besar penelitian deteksi ambiguitas *requirement* berfokus pada dokumen berbahasa Inggris, sementara ambiguitas pada dokumen berbahasa Indonesia—sebagai bahasa dengan sumber daya terbatas (*low-resource language*)—hampir tidak tersentuh Bashir et al. (2025). Bahasa Indonesia memiliki pola ambiguitas tersendiri, seperti ketidakhadiran penanda waktu (*tense*), ambiguitas afiksasi, dan redundansi kata, yang membuat hasil penelitian berbasis bahasa Inggris tidak serta-merta dapat digeneralisasi. Oleh karena itu, penelitian ini juga mengevaluasi efektivitas arsitektur yang diusulkan pada dokumen *requirement* berbahasa Indonesia dan Inggris untuk mengidentifikasi perbedaan lintas bahasa.

Berdasarkan identifikasi kesenjangan tersebut, penelitian ini mengusulkan arsitektur *Role-Based Multi-Agent LLM* yang mengorkestrasi agen-agen dengan perspektif berbeda—*stakeholder*, *developer*, dan *tester*—untuk mendeteksi ambiguitas *requirement* perangkat lunak secara lebih komprehensif. Evaluasi dilakukan secara komparatif terhadap *single-agent LLM* dan *Self-MoA*, serta meliputi analisis berdasarkan taksonomi ambiguitas dan studi lintas bahasa (Indonesia dan Inggris).

1.2 Perumusan Masalah

Berdasarkan uraian latar belakang, perumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana arsitektur *role-based multi-agent LLM* dapat dirancang untuk mendeteksi ambiguitas *requirement* perangkat lunak secara lebih komprehensif dibandingkan pendekatan *single-agent LLM*?
2. Sejauh mana spesialisasi peran (*role-based*) memberikan peningkatan performa deteksi ambiguitas dibandingkan dengan konfigurasi *Self-MoA* (model yang sama diulang) dan *single-agent*?
3. Bagaimana efektivitas arsitektur *role-based multi-agent LLM* dalam mendeteksi berbagai kategori ambiguitas berdasarkan taksonomi ambiguitas (leksikal, sintaktis, semantik, kekaburan, ketidaklengkapan, dan kontradiksi)?
4. Bagaimana perbedaan bahasa (*Indonesia vs. Inggris*) memengaruhi kemampuan deteksi ambiguitas *requirement* oleh arsitektur yang diusulkan?

1.3 Batasan Penelitian

Untuk memastikan fokus dan keterlaksanaan penelitian, batasan penelitian ditetapkan sebagai berikut:

1. Objek penelitian dibatasi pada dokumen *requirement* perangkat lunak dalam format teks naratif (*natural language*), bukan model formal (seperti UML atau *formal specification*).
2. Evaluasi ambiguitas menggunakan taksonomi yang mencakup enam kategori: leksikal, sintaktis, semantik, kekaburan, ketidaklengkapan, dan kontradiksi Berry and Kamsties (2004).
3. Agen yang digunakan dalam arsitektur *role-based multi-agent* dibatasi pada tiga peran: *stakeholder*, *developer*, dan *tester*.
4. Perbandingan komparatif dilakukan terhadap tiga konfigurasi: (1) *single-agent LLM*, (2) *Self-MoA*, dan (3) *role-based multi-agent*.
5. Studi bahasa dibatasi pada perbandingan dokumen berbahasa Indonesia dan Inggris.
6. Model LLM yang digunakan mencakup model *open-source* (melalui Ollama) dan/atau *closed-source* (melalui API), sesuai ketersediaan dan keterjangkauan.

1.4 Tujuan Penelitian

Tujuan utama dari penelitian ini adalah:

1. Merancang arsitektur *role-based multi-agent LLM* untuk deteksi ambiguitas *requirement* perangkat lunak.
2. Mengevaluasi secara komparatif performa deteksi ambiguitas antara arsitektur *role-based multi-agent*, *Self-MoA*, dan *single-agent LLM* menggunakan metrik *precision*, *recall*, dan F1.
3. Menganalisis kontribusi tiap peran (*stakeholder*, *developer*, *tester*) terhadap deteksi kategori ambiguitas spesifik berdasarkan taksonomi ambiguitas.
4. Mengidentifikasi perbedaan efektivitas deteksi ambiguitas pada dokumen *requirement* berbahasa Indonesia dibandingkan bahasa Inggris.

1.5 Manfaat Penelitian

Hasil dari penelitian ini diharapkan dapat memberikan manfaat:

1. **Bagi Praktisi Rekayasa Perangkat Lunak:** Menyediakan pendekatan yang lebih komprehensif untuk mendeteksi ambiguitas *requirement* sejak tahap awal pengembangan, sehingga mengurangi risiko *rework* dan kegagalan proyek.
2. **Bagi Komunitas Riset Requirements Engineering:** Menjadi referensi pertama yang menguji arsitektur *role-based multi-agent LLM* secara spesifik untuk deteksi ambiguitas, mengisi kesenjangan yang belum ditangani oleh Oriol et al. (2025), Bashir et al. (2025), dan Jin et al. (2025).
3. **Bagi Komunitas Riset Multi-Agent LLM:** Memberikan bukti empiris mengenai apakah spesialisasi peran (*role-based*) memberikan keunggulan signifikan dibandingkan *Self-MoA*, merespons temuan Li et al. (2025).
4. **Bagi Pengembang di Indonesia:** Menawarkan evaluasi deteksi ambiguitas pada dokumen berbahasa Indonesia, yang hampir tidak memiliki penelitian sebelumnya, sehingga mendukung praktik RE yang lebih baik di konteks lokal.

Bab II

PENELITIAN TERKAIT

2.1 Studi Terkait

Penelitian ini beririsan dengan beberapa alur riset yang telah berkembang dalam beberapa tahun terakhir. Berikut adalah tinjauan terhadap penelitian-penelitian kunci yang relevan, beserta identifikasi kesenjangan (*gap*) yang dapat diisi oleh penelitian ini.

2.1.1 Deteksi Ambiguitas Requirement Menggunakan Single-Agent LLM

Bashir et al. (2025) mengadakan studi industri yang dipresentasikan pada ICS-ME 2025 mengenai deteksi dan penjelasan ambiguitas *requirement* menggunakan LLM Bashir et al. (2025). Penelitian ini menunjukkan bahwa *10-shot prompting* pada LLM *single-agent* mampu meningkatkan performa deteksi ambiguitas sebesar +20,2% dibandingkan *zero-shot*, dengan skor persetujuan manusia (*human agreement*) sebesar 3,84 dari 5. Meskipun hasil ini menjanjikan, pendekatan *single-agent* secara inheren terbatas pada satu perspektif analisis—LLM membaca *requirement* dari satu sudut pandang dan tidak dapat secara alami mengadopsi lensa berbeda yang diperlukan untuk menangkap seluruh kategori ambiguitas. Penelitian ini juga tidak mengevaluasi dampak orkestrasi multi-agent terhadap performa deteksi.

Vijayvargiya et al. (2025) mengusulkan *Ambig-SWE*, yang mengevaluasi agen interaktif untuk mengatasi ambiguitas dalam tugas *code generation* Vijayvargiya et al. (2025). Claude Sonnet mencapai akurasi 84% pada tugas yang diberikan. Namun, fokus penelitian ini pada *code generation*—bukan analisis *requirement*—dan penggunaan agen interaktif tanpa spesialisasi peran membuatnya tidak secara langsung dapat dibandingkan dengan deteksi ambiguitas *requirement*.

2.1.2 Multi-Agent LLM untuk Requirements Engineering

Oriol et al. (2025) merupakan penelitian pertama yang mengaplikasikan strategi *Multi-Agent Debate* (MAD) pada domain RE Oriol et al. (2025). Penelitian ini menguji klasifikasi biner fungsional vs. non-fungsional dengan peningkatan F1 sebesar +10,9% dibandingkan *baseline*, namun biaya komputasi meningkat 16 kali lipat. Pendekatan mereka bersifat *debater-vs-debater* tanpa spesialisasi peran, dan belum diaplikasikan pada tugas deteksi ambiguitas. Analisis biaya yang dilakukan masih bersifat kasar (*rough*), tanpa *ablation study* yang memisahkan kontribusi tiap komponen.

Jin et al. (2025) mengusulkan iReDev, kerangka kerja berbasis pengetahuan dengan 6 agen berbasis peran—*interviewer*, *end-user*, *deployer*, *analyst*, *archivist*, *reviewer*—untuk siklus RE end-to-end Jin et al. (2025). Meskipun arsitektur ini mendemonstrasikan konsep agen berspesialisasi peran, fokusnya pada siklus RE penuh

(*full lifecycle*) membuatnya tidak mengevaluasi secara mendalam kontribusi tiap peran terhadap deteksi ambiguitas secara individual.

Cheng et al. (2026) mengembangkan QUARE, yang menggunakan agen terkualitas (*quality-specialized agents*) dengan negosiasi dialektis untuk menyeimbangkan atribut kualitas dalam RE Cheng et al. (2026). Sistem ini mencapai kepatuhan 98,2%, namun fokusnya pada *quality attributes* bukan pada ambiguitas, sehingga mekanisme negosiasi yang dikembangkan tidak dapat langsung ditransfer ke domain deteksi ambiguitas.

Lu et al. (2026) membangun ReDeFo, arsitektur multi-agent dengan agen *Analyst*, *Formalizer*, dan *Coder* untuk transformasi *requirement* menjadi kode terverifikasi Lu et al. (2026). Fokus pada formalisasi—bukan identifikasi ambiguitas—membuat pendekatan ini berada di domain yang berbeda, meskipun arsitektur multi-agent yang digunakan relevan sebagai perbandingan.

2.1.3 Multi-Agent LLM di Domain Lain yang Relevan

Giroh et al. (2025) dari Amazon mengusulkan kerangka kerja multi-agent dengan tiga agen (*Clarifier*, *Planner*, *Implementor*) untuk mentransformasikan SOP ambigu menjadi kode Giroh et al. (2025). Sistem ini mencapai akurasi 88,4% pada domain SOP. Meskipun domainnya berbeda (SOP vs. *software requirement*), pendekatan yang mirip—menggunakan agen dengan peran khusus untuk mengatasi ambiguitas—menjadi referensi arsitektur yang relevan.

2.1.4 Fondasi Multi-Agent LLM dan Tantangan Self-MoA

Wang et al. (2024) mengusulkan kerangka kerja *Mixture-of-Agents* (MoA) yang mengorkestrasi beberapa LLM secara berlapis untuk meningkatkan kapabilitas Wang et al. (2024). MoA menunjukkan peningkatan signifikan pada berbagai *benchmark*, membuka jalan bagi arsitektur multi-agent di berbagai domain.

Li et al. (2025) menemukan bahwa konfigurasi *Self-MoA*—di mana model yang sama diulang beberapa kali sebagai agen yang berbeda—seringkali mengungguli *Mixed-MoA* yang menggunakan model yang berbeda-beda Li et al. (2025). Temuan ini merupakan tantangan kritis bagi penelitian multi-agent: jika repetisi model yang sama sudah mencukupi, maka kompleksitas tambahan dari spesialisasi peran perlu dibuktikan memberikan nilai tambah yang signifikan. Penelitian ini merespons tantangan tersebut dengan membandingkan secara langsung *role-based multi-agent* versus *Self-MoA*.

Du et al. (2023) memperkenalkan *multiagent debate* sebagai mekanisme untuk meningkatkan faktualitas dan penalaran dalam model bahasa Du et al. (2023). Prinsip bahwa interaksi dan debat antar-agen dapat menghasilkan jawaban yang lebih baik menjadi fondasi teoretis bagi pendekatan multi-agent dalam penelitian ini.

Tabel 2.1: Perbandingan Penelitian Terkait

Penelitian	Multi-Agent	Role-Based	Ambiguitas Req.	Bahasa Indonesia	Komparatif vs. Self-MoA
Bashir et al. (2025)	×	×	✓	×	×
Vijayvargiya et al. (2025)	✓*	×	×	×	×
Oriol et al. (2025)	✓	×	×	×	×
Jin et al. (2025)	✓	✓	×	×	×
Cheng et al. (2026)	✓	✓	×	×	×
Lu et al. (2026)	✓	✓	×	×	×
Giroh et al. (2025)	✓	✓	×	×	×
Penelitian ini	✓	✓	✓	✓	✓

✓*: Agen interaktif untuk *code generation*, bukan arsitektur multi-agent untuk RE.

2.1.5 Sintesis dan Identifikasi Gap

Berdasarkan sintesis pada Tabel 2.1, teridentifikasi bahwa **belum ada penelitian yang secara spesifik menggabungkan arsitektur role-based multi-agent LLM dengan deteksi ambiguitas requirement perangkat lunak**, dan belum ada yang melakukan studi komparatif terhadap *Self-MoA* dalam konteks ini. Penelitian ini mengisi kesenjangan tersebut.

Bab III

METODE DAN RANCANGAN PENELITIAN

3.1 Rancangan Penelitian

Penelitian ini menggunakan rancangan *eksperimental komparatif* yang bertujuan mengevaluasi efektivitas arsitektur *role-based multi-agent LLM* dalam mendeteksi ambiguitas *requirement* perangkat lunak. Rancangan penelitian terdiri dari beberapa tahap yang diuraikan pada sub-bab berikut.

3.1.1 Tahap 1: Pengumpulan dan Preparasi Dataset

Dataset yang digunakan dalam penelitian ini terdiri dari dua sumber:

1. **Dataset berbahasa Inggris:** Diambil dari *public dataset* yang telah tersedia dalam literatur RE, seperti PROMISE Repository dan dataset yang digunakan dalam Bashir et al. (2025) Bashir et al. (2025). Dataset ini telah memiliki label ambiguitas yang dapat diverifikasi.
2. **Dataset berbahasa Indonesia:** Dikonstruksi melalui proses translasi dan augmentasi dari dataset berbahasa Inggris, kemudian divalidasi oleh *annotator* yang memahami konteks RE dan bahasa Indonesia. Jika memungkinkan, dataset ini juga diperkaya dengan dokumen *requirement* dari proyek-proyek perangkat lunak berbahasa Indonesia yang bersifat *open-source*.

Setiap *requirement* dalam dataset dilabeli berdasarkan taksonomi ambiguitas: leksikal, sintaktis, semantik, kekaburan (*vagueness*), ketidaklengkapan (*incompleteness*), dan kontradiksi Berry and Kamsties (2004). Label dapat bersifat *multi-label* karena satu *requirement* dapat mengandung lebih dari satu jenis ambiguitas.

3.1.2 Tahap 2: Perancangan Arsitektur Role-Based Multi-Agent LLM

Arsitektur yang diusulkan terdiri dari tiga agen utama dengan peran yang berbeda:

1. **Agen Stakeholder:** Menganalisis *requirement* dari perspektif pengguna akhir dan pihak berkepentingan. Agen ini berfokus pada deteksi ambiguitas yang memengaruhi pemahaman kebutuhan bisnis, seperti kekaburan (*vagueness*) dan ketidaklengkapan (*incompleteness*).
2. **Agen Developer:** Menganalisis *requirement* dari perspektif implementasi teknis. Agen ini berfokus pada deteksi ambiguitas yang memengaruhi desain dan pengkodean, seperti ambiguitas leksikal, sintaktis, dan semantik.

3. **Agen Tester:** Menganalisis *requirement* dari perspektif pengujian dan verifikasi. Agen ini berfokus pada deteksi ambiguitas yang memengaruhi kejelasan kriteria pengujian, seperti kontradiksi dan ketidaklengkapan.

Mekanisme orkestrasi mengikuti kerangka kerja *Mixture-of-Agents* (MoA) Wang et al. (2024) dengan modifikasi berbasis peran:

- Setiap agen menerima *requirement* dan menghasilkan analisis ambiguitas secara independen.
- Hasil analisis dari ketiga agen diintegrasikan oleh *aggregator* yang mengkonsolidasi temuan, menghilangkan duplikasi, dan menghasilkan laporan ambiguitas terintegrasi.
- Jika terdapat konflik antar-agen (misalnya, satu agen menganggap suatu aspek ambigu sementara agen lain tidak), mekanisme *voting* atau *debate* diaktifkan untuk mencapai konsensus.

3.1.3 Tahap 3: Konfigurasi Komparatif

Evaluasi dilakukan dengan membandingkan tiga konfigurasi:

1. **Single-Agent LLM:** Satu LLM tanpa orkestrasi multi-agent, digunakan sebagai *baseline*. Konfigurasi ini mereproduksi pendekatan Bashir et al. (2025).
2. **Self-MoA:** Model LLM yang sama digunakan sebanyak n kali sebagai agen yang identik, tanpa spesialisasi peran. Konfigurasi ini merespons temuan Li et al. (2025) bahwa repetisi model seringkali mengungguli model heterogen Li et al. (2025).
3. **Role-Based Multi-Agent:** Arsitektur yang diusulkan dengan tiga agen berspesialisasi peran (*stakeholder*, *developer*, *tester*).

Seluruh konfigurasi diuji pada dataset yang sama dengan parameter yang setara (*temperature*, *max tokens*, dll.) untuk memastikan validitas perbandingan.

3.1.4 Tahap 4: Pengukuran dan Evaluasi

Metrik evaluasi yang digunakan mencakup:

- **Metrik deteksi:** *Precision*, *Recall*, dan F1-Score untuk setiap kategori ambiguitas dan secara keseluruhan.
- **Metrik persetujuan manusia:** *Inter-Annotator Agreement* (Cohen's κ atau Krippendorff's α) antara hasil deteksi sistem dan penilaian *annotator* manusia.

- **Metrik biaya:** Jumlah *token* yang dikonsumsi dan waktu eksekusi per konfigurasi, untuk mengukur *cost-effectiveness* relatif.

Ablation study dilakukan untuk:

1. Mengukur kontribusi tiap peran (*stakeholder*, *developer*, *tester*) terhadap performa keseluruhan.
2. Menganalisis sensitivitas terhadap jumlah agen dan kedalaman orkestrasi.
3. Membandingkan dampak penggunaan model *open-source* vs. *closed-source* sebagai *backbone* tiap agen.

3.1.5 Tahap 5: Analisis Lintas Bahasa

Pengujian dilakukan pada dataset berbahasa Indonesia dan Inggris untuk mengidentifikasi:

- Perbedaan distribusi kategori ambiguitas antar-bahasa.
- Perbedaan efektivitas deteksi oleh tiap konfigurasi pada kedua bahasa.
- Pola ambiguitas spesifik bahasa Indonesia yang tidak tercakup dalam taksonomi berbasis bahasa Inggris.

3.2 Alat dan Bahan

1. **Model LLM:** Model *open-source* yang diakses melalui platform Ollama Cloud Pro dengan langganan aktif, mencakup model *cloud* berikut:
 - *DeepSeek V4 Pro* (671B MoE, 49B aktif): Model reasoning terkuat dengan tiga mode penalaran (low/medium/high), digunakan sebagai *backbone* utama untuk konfigurasi *role-based multi-agent* dan *Self-MoA*.
 - *Qwen 3.5* (122B MoE, 10B aktif): Model multibahasa dengan kemampuan reasoning kuat, digunakan sebagai agen *stakeholder* yang memerlukan pemahaman konteks lintas bahasa (Indonesia dan Inggris).
 - *GPT-OSS 120B* (120B MoE, 5.1B aktif): Model *open-weight* dari OpenAI dengan arsitektur MoE yang berbeda dari DeepSeek dan Qwen, digunakan sebagai agen *tester* untuk meningkatkan keragaman arsitektur dalam konfigurasi *Mixed-MoA*.
 - *Qwen 3.5* varian kecil (27B dan 9B): Digunakan untuk *ablation study* terhadap ukuran model.

Seluruh model *cloud* diakses melalui API Ollama yang kompatibel dengan OpenAI, sehingga pergantian antar model hanya memerlukan perubahan satu baris konfigurasi. Model-model ini juga tersedia secara *open-weight* di HuggingFace, Ollama lokal, OpenRouter, dan Together AI, memungkinkan reproduksibilitas oleh peneliti lain tanpa biaya langganan.

2. **Kerangka kerja Multi-Agent:** CrewAI (v0.86+), kerangka kerja Python berbasis peran (*role-based*) yang mendukung definisi agen dengan *role*, *goal*, dan *backstory* secara natif. CrewAI terintegrasi dengan Ollama melalui LiteLLM, mendukung orkestrasi sekuensial dan hierarkis, serta menyediakan mekanisme delegasi tugas antar-agen. Sebagai *baseline* MoA tanpa peran, digunakan implementasi referensi Together MoA (~200 baris Python) yang dimodifikasi untuk kompatibilitas dengan Ollama.
3. **Dataset:**
 - *Dataset berbahasa Inggris:* Orchid Benchmark (1.304 tugas *requirement* berlabel ambiguitas) dan augmentasi dari Mendeley Requirements Ambiguity Dataset, dengan label tambahan untuk kategori ambiguitas yang belum tercakup (kekaburan dan kontradiksi).
 - *Dataset berbahasa Indonesia:* Dikonstruksi melalui translasi dan augmentasi dari dataset berbahasa Inggris, divalidasi oleh *annotator* yang memahami konteks RE dan bahasa Indonesia.
4. **Lingkungan Komputasi:** Ollama Cloud Pro sebagai infrastruktur utama untuk inferensi model *cloud*, dengan GPU lokal sebagai cadangan untuk model berukuran kecil (7B–14B) selama tahap pengembangan dan iterasi cepat. Pelacakan konsumsi *token* dilakukan melalui metrik *prompt_eval_count* dan *eval_count* yang disediakan oleh API Ollama.

Bab IV

JADWAL PENELITIAN

4.1 Rencana Kerja

Jadwal penelitian berikut disusun sebagai contoh timeline bergaya Gantt chart yang bisa disesuaikan dengan target durasi penelitian.

	April 2026				Mei 2026				Juni 2026				Juli 2026			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Persiapan																
Tinjauan pustaka																
Analisis model																
Penyusunan proposal																
Pengujian proposal																
Implementasi																
Implementasi model																
Ideasi produk																
Penelitian dan desain UI/UX																
Perancangan																
Pengujian																
Penyusunan laporan akhir																

Blok biru menunjukkan periode aktivitas utama. Bulan dan minggu bisa diganti sesuai durasi penelitian yang sebenarnya.

DAFTAR PUSTAKA

- Bashir, S., Ferrari, A., Khan, A., Strandberg, P., Haider, Z., Saadatmand, M., and Bohlin, M. (2025). Requirements ambiguity detection and explanation with LLMs: An industrial study. In *Proc. IEEE Int. Conf. Softw. Maintenance Evol. (ICSME 2025)*, Malmö. IEEE.
- Berry, D. M. and Kamsties, E. (2004). Ambiguity in requirements specification. In *Proc. 1st Int. Workshop Perspect. Softw. Requirements*, pages 7–12, Auckland.
- Berry, D. M., Kamsties, E., and Krieger, M. M. (2000). From the start: A primer on requirements ambiguity. *ACM SIGSOFT Softw. Eng. Notes*, 25(4):12–13.
- Cheng, H., Kim, M., Khomh, F., Racharak, T., Yoshioka, N., Ubayashi, N., and Washizaki, H. (2026). QUARE: Multi-agent negotiation for balancing quality attributes in requirements engineering. arXiv:2603.11890, diakses 14 Juni 2026.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J. B., and Mordatch, I. (2023). Improving factuality and reasoning in language models through multiagent debate. arXiv:2305.14325, diakses 14 Juni 2026.
- Ferrara, F., Fattoruso, G., and De Rosa, M. (2014). A natural language inference approach for ambiguity detection in requirements specification. In *Proc. 2014 IEEE Int. Requirements Eng. Conf.*, pages 303–308, Karlskrona. IEEE.
- Giroh, S. K., Ghosh, P., Jain, A., Paunikar, H. G., Rastogi, A., Yenigalla, P., and Nediychath, A. (2025). Structuring the unstructured: A multi-agent LLM framework for transforming ambiguous SOPs into code. In *Proc. 2025 Conf. Empir. Methods Nat. Lang. Process. Ind. Track*, Miami. ACL.
- Jin, D., Sun, W., Huang, J., Liang, P., Xuan, J., Liu, Y., and Jin, Z. (2025). iReDev: A knowledge-driven multi-agent framework for intelligent requirements development. *ACM Trans. Softw. Eng. Methodol.*
- Kamsties, E., Berry, D. M., and Paech, B. (1998). Detecting ambiguity in requirements: An empirical comparison. *Requirements Engineering*, 3(3):192–209.
- Li, W., Lin, Y., Xia, M., and Jin, C. (2025). Rethinking mixture-of-agents: Is mixing different LLMs beneficial? arXiv:2502.00674, diakses 14 Juni 2026.
- Lu, X., Sun, W., Zhang, Y., Hu, M., Tian, C., Jin, Z., and Liu, Y. (2026). ReDeFo: Requirements development and formalization for reliable code generation: A multi-agent vision. arXiv:2508.18675, diakses 14 Juni 2026.
- Oriol, M., Motger, Q., Marco, J., and Franch, X. (2025). Multi-agent debate strategies to enhance requirements engineering with large language models. arXiv:2507.05981, diakses 14 Juni 2026.

- The Standish Group (1995). *CHAOS Report*. The Standish Group International, West Yarmouth.
- Vijayvargiya, S., Zhou, X., Yerukala, A., Sap, M., and Neubig, G. (2025). Ambig-SWE: Interactive agents to overcome ambiguity in software engineering. arXiv:2502.13069, presented at ICLR 2026, diakses 14 Juni 2026.
- Wang, J., Wang, J., Athiwaratkun, B., Zhang, C., and Zou, J. (2024). Mixture-of-agents enhances large language model capabilities. In *Proc. Int. Conf. Learn. Represent. (ICLR 2025)*, Singapore.
- Wieggers, K. E. (2003). *Software Requirements*. Microsoft Press, Redmond, 2 edition.